Universidad Nacional del Altiplano — Puno
Escuela Profesional de Ingeniería de Sistemas

Informe de Práctica

Semana 9: Árboles Binarios de Búsqueda (BST)

Algoritmos y Estructuras de Datos
Periodo académico 2026-II

Estudiante: Angheli Luana Cuevas Yucra
Código: 251078
Ciclo: 4to
Docente: Dr. Aldo Hernán Zanabria Gálvez
Modalidad: Individual
Fecha: 27/10/26

Puno, Perú

1. Introducción y objetivos

Buscar un código estudiantil recorriendo toda una lista toma tiempo proporcional a la cantidad de registros. El Sistema Integrado de Gestión Académica (SIGA) de la UNAP maneja alrededor de 45 000 estudiantes activos, por lo que se necesita una estructura que permita buscar, insertar y eliminar con pocos pasos. Un árbol binario de búsqueda (BST) es una alternativa: en cada comparación descarta una parte del árbol.

El objetivo de esta práctica es implementar un BST de estudiantes cuya clave es el código UNAP, y comprobar con mediciones por qué su rendimiento depende de la altura del árbol. Concretamente:

- implementar inserción, búsqueda, recorrido inorden y eliminación en C++17.
- medir la altura del árbol con inserción en orden y con inserción aleatoria;
- simular el módulo de consulta del SIGA-UNAP con un menú interactivo en Python.

2. Fundamento teórico

Un BST es un árbol binario donde, para cada nodo, todas las claves del subárbol izquierdo son menores y todas las del subárbol derecho son mayores (Cormen et al., 2022). Por esa propiedad, buscar una clave consiste en comparar con el nodo actual y avanzar a la izquierda o a la derecha. El recorrido inorden (izquierda, nodo, derecha) entrega las claves ordenadas de menor a mayor.

Búsqueda, inserción y eliminación siguen un camino de la raíz hacia una hoja, de modo que su costo es $O(h)$, donde h es la altura del árbol. Si el árbol está razonablemente equilibrado, $h = O(\log n)$. Pero si las claves llegan en orden ascendente o descendente, el árbol se vuelve una cadena con $h = n - 1$ y las operaciones pasan a costar $O(n)$. Para n claves insertadas en orden aleatorio, la altura esperada es aproximadamente $1{,}39 \log_2 n$ (Sedgewick y Wayne, 2011). Los árboles AVL y Rojo-Negro evitan la degeneración manteniendo $h = O(\log n)$ en el peor caso.

En este informe la altura se mide en *aristas*: un árbol vacío tiene altura -1, un árbol de un solo nodo tiene altura 0 y una cadena de n nodos tiene altura $n - 1$.

3. Desarrollo

3.1. Actividad 1: BST desde cero en C++17

El programa `bst_unap.cpp` guarda estudiantes con código, nombre y escuela. Para mantener el código simple y fácil de explicar, no se usó la librería estándar: los textos son arreglos `char`, los nodos se enlazan con punteros comunes, se comparan códigos con `strcmp` y se imprime con `printf`. El destructor libera la memoria recorriendo el árbol en postorden.

Código 1: Inserción en el BST (`insertar_aux`).

```
void insertar_aux(Nodo*& nodo, const Estudiante& e) {
    if (nodo == nullptr) {            // hueco encontrado: crear nodo aqui
        nodo = new Nodo(e);
        cantidad_++;
        return;
    }
```

Cuadro 1: Funciones principales de la Actividad 1.

Función	Qué hace	Costo
`insertar_aux`	Baja comparando códigos hasta hallar un hueco y ahí crea el nodo (siempre como hoja). Si el código ya existe, no lo repite.	$O(h)$
`buscar_aux`	Si el código es menor que el del nodo va a la izquierda, si es mayor a la derecha; devuelve el nodo o `nullptr`.	$O(h)$
`inorden_aux`	Recorre izquierda, nodo, derecha y guarda los estudiantes en un arreglo, ya ordenados.	$O(n)$

```
    int c = strcmp(e.codigo, nodo->dato.codigo);
    if (c < 0)      insertar_aux(nodo->izq, e);   // menor -> izquierda
    else if (c > 0) insertar_aux(nodo->der, e);   // mayor -> derecha
    // c == 0: codigo repetido, no hacemos nada
}
```

Código 2: Búsqueda recursiva (`buscar_aux`).

```
Nodo* buscar_aux(Nodo* nodo, const char* cod) const {
    if (nodo == nullptr) return nullptr;          // no esta
    int c = strcmp(cod, nodo->dato.codigo);
    if (c == 0) return nodo;                      // lo encontre
    if (c < 0)  return buscar_aux(nodo->izq, cod);
    return buscar_aux(nodo->der, cod);
}
```

Código 3: Recorrido inorden (`inorden_aux`).

```
void inorden_aux(Nodo* nodo, Estudiante* resultado, int& n) const {
    if (nodo == nullptr) return;
    inorden_aux(nodo->izq, resultado, n);   // 1) todo lo menor
    resultado[n] = nodo->dato;              // 2) yo
    n++;
    inorden_aux(nodo->der, resultado, n);   // 3) todo lo mayor
}
```

El parámetro `Nodo*&` de `insertar_aux` permite modificar el puntero del padre y así colgar el nodo nuevo en el hueco encontrado.

3.2. Actividad 2A: Eliminación

La eliminación distingue tres casos según los hijos del nodo a borrar:

1. **Hoja (sin hijos):** se elimina el nodo y el puntero del padre queda en `nullptr`.
2. **Un hijo:** el hijo ocupa el lugar del nodo eliminado.
3. **Dos hijos:** se toma el sucesor inorden (el menor del subárbol derecho), se copian sus datos al nodo y se elimina el sucesor original, que tiene a lo sumo un hijo.

Código 4: Eliminación con los tres casos (`eliminar_aux`).

```
    void eliminar_aux(Nodo*& nodo, const char* cod) {
        if (nodo == nullptr) return;                    // no existe
        int c = strcmp(cod, nodo->dato.codigo);
        if (c < 0) {
            eliminar_aux(nodo->izq, cod);
        } else if (c > 0) {
            eliminar_aux(nodo->der, cod);
        } else {
            // Lo encontre. Veo cuantos hijos tiene:
            if (nodo->izq == nullptr) {
                // Caso 1 (hoja) y caso 2 (solo hijo derecho):
                // el hijo derecho (o nada) ocupa su lugar
                Nodo* viejo = nodo;
                nodo = nodo->der;
                delete viejo;
                cantidad_--;
            } else if (nodo->der == nullptr) {
                // Caso 2: solo hijo izquierdo, sube ese hijo
                Nodo* viejo = nodo;
                nodo = nodo->izq;
                delete viejo;
                cantidad_--;
            } else {
                // Caso 3: dos hijos. Tomo el sucesor (el menor de la derecha),
                // copio sus datos aqui y borro el sucesor original.
                Nodo* suc = minimo(nodo->der);
                nodo->dato = suc->dato;
                eliminar_aux(nodo->der, nodo->dato.codigo);
            }
        }
    }
```

Se agregaron las pruebas P4 (eliminar una hoja), P5 (eliminar la raíz, que tiene dos hijos) y P6 (el árbol sigue siendo un BST válido con 3 elementos). La ejecución, compilada con `g++ -std=c++17 -Wall -Wextra`, produjo:

Código 5: Salida de `bst_unap`.

```
[P1] OK
[P2] OK
[P3] OK
Listado ordenado:
  UNAP-2021-000020 - Huanca Puma, Lucia
  UNAP-2022-000030 - Flores Condori, Ana
  UNAP-2023-000040 - Ccopa Ramos, Pedro
  UNAP-2023-000050 - Mamani Quispe, Juan
  UNAP-2024-000070 - Ticona Apaza, Carlos
[P4] OK
[P5] OK
[P6] OK
```

El programa también se ejecutó con AddressSanitizer y UBSan sin reportar errores de memoria.

3.3. Actividad 2B: Análisis de degeneración

El programa `deg_analysis.py` construye árboles con $N \in \{10, 20, 50, 100\}$ códigos de la forma `UNAP-0001`, `UNAP-0002`, ... En cada caso:

- inserta los códigos en orden ascendente y mide la altura (h_{ord});
- los inserta mezclados al azar, repite 5 veces y promedia la altura (h_{ale}); se fijó la semilla 42 para que los resultados sean reproducibles;
- calcula la altura teórica $1{,}39 \log_2 N$ y la razón h_{ord}/h_{ale}.

Código 6: Inserción y altura en `deg_analysis.py`.

```
def insertar(raiz, clave):
    if raiz is None:
        return Nodo(clave)
    if clave < raiz.clave:
        raiz.izq = insertar(raiz.izq, clave)
    elif clave > raiz.clave:
        raiz.der = insertar(raiz.der, clave)
    return raiz


def altura(raiz) -> int:
    if raiz is None:
        return -1
    return 1 + max(altura(raiz.izq), altura(raiz.der))
```

Cuadro 2: Altura del BST según el orden de inserción.

N	h (orden ascendente)	h (aleatorio prom.)	h teórico $1{,}39 \log_2 N$	Razón h_{ord}/h_{ale}
10	9	4,6	4,6	2,0
20	19	6,6	6,0	2,9
50	49	8,8	7,8	5,6
100	99	12,4	9,2	8,0

Análisis. Con inserción en orden ascendente la altura es siempre $N - 1$: el árbol es una cadena y buscar el último código requiere recorrer todos los nodos, igual que en una lista. Con inserción aleatoria la altura crece mucho más despacio, siguiendo el orden de $\log_2 N$; los valores observados quedan algo por encima de la fórmula teórica, que es una aproximación, pero el crecimiento logarítmico es claro. La razón entre ambas alturas pasa de 2,0 a 8,0 cuando N crece de 10 a 100, y seguiría aumentando con N.

Conclusión. El BST simple no es adecuado para el SIGA-UNAP si los códigos estudiantiles se insertan en orden ascendente, porque el árbol degenera en una cadena y las búsquedas pasan a costar $O(n)$, justo lo que se quería evitar. La estructura que resuelve este problema es el árbol AVL, que se estudiará en la Semana 10 y mantiene la altura en $O(\log n)$ mediante rotaciones.

3.4. Actividad 3: Sistema de consulta SIGA-UNAP

El programa `siga_consulta.py` simula el módulo de consulta del SIGA-UNAP. Al iniciar carga 20 estudiantes generados al azar, con códigos distintos entre sí. Ofrece el menú: [1] Buscar

por código, [2] Listar en orden, [3] Insertar estudiante, [4] Eliminar por código y [5] Salir. Cada operación muestra su tiempo medido con `time.perf_counter()`; el tiempo se toma solo alrededor de la operación, sin contar lo que tarda el usuario en escribir. Al salir informa el total de operaciones y la altura final del árbol.

La clase `SIGA_BST` implementa la inserción, la búsqueda, el listado (recorrido inorden), la eliminación con los tres casos y el cálculo de la altura. La inserción y la eliminación se escribieron de forma recursiva:

Código 7: Inserción en `SIGA_BST`.

```
    def _insertar(self, nodo, est):
        if nodo is None:                        # hueco: aqui va el nuevo nodo
            return Nodo(est)
        if est.codigo < nodo.dato.codigo:
            nodo.izq = self._insertar(nodo.izq, est)
        elif est.codigo > nodo.dato.codigo:
            nodo.der = self._insertar(nodo.der, est)
        return nodo                             # si es igual, no se repite
```

Código 8: Eliminación en `SIGA_BST`.

```
    def _eliminar(self, nodo, codigo):
        if nodo is None:
            return None, False
        if codigo < nodo.dato.codigo:
            nodo.izq, ok = self._eliminar(nodo.izq, codigo)
            return nodo, ok
        if codigo > nodo.dato.codigo:
            nodo.der, ok = self._eliminar(nodo.der, codigo)
            return nodo, ok
        # Encontrado: tres casos
        if nodo.izq is None:                    # hoja o solo hijo derecho
            return nodo.der, True
        if nodo.der is None:                    # solo hijo izquierdo
            return nodo.izq, True
        # dos hijos: el sucesor es el menor del lado derecho
        suc = nodo.der
        while suc.izq is not None:
            suc = suc.izq
        nodo.dato = suc.dato
        nodo.der, _ = self._eliminar(nodo.der, suc.dato.codigo)
        return nodo, True
```

Se probó el menú con una sesión de ejemplo (extracto). Los tiempos dependen del equipo y varían en cada ejecución.

Código 9: Extracto de una sesión de `siga_consulta.py`.

```
SIGA-UNAP iniciado con 20 registros. Altura del árbol: 7

[1] Buscar  [2] Listar  [3] Insertar  [4] Eliminar  [5] Salir
Opción: 1
Código: UNAP-0000-000000
No encontrado
Tiempo: 0.002 ms
```

```
Opción: 3        (se inserta UNAP-2024-000001)
Insertado
Tiempo: 0.004 ms

Opción: 1        (se busca UNAP-2024-000001)
Perez Gomez, M. (Ing. Sistemas)
Tiempo: 0.002 ms

Opción: 4        (se elimina UNAP-2024-000001)
Eliminado
Tiempo: 0.006 ms

Opción: 4        (se repite la eliminación)
No encontrado
Tiempo: 0.003 ms

Opción: 5
Total operaciones: 5
Altura final del árbol: 7
```

4. Reflexión final

¿Cuántas comparaciones se necesitan para buscar a UNAP-2024-045000 en un padrón de 45 000 estudiantes?

Cuadro 3: Comparaciones estimadas para un padrón de 45 000 estudiantes.

Estructura	**Comparaciones**	**Cómo se obtiene**
BST con inserción en orden	$\approx 45\,000$	el código buscado es el último de la cadena
AVL	≤ 22 (ideal: 16)	$1{,}44 \log_2 45\,000 \approx 22$; $\lceil \log_2 45\,001 \rceil = 16$
BST aleatorio	1 en el mejor caso; ≈ 21 lo normal	mejor caso: la clave está en la raíz; típico: $1{,}39 \log_2 45\,000$

La diferencia entre unas 45 000 comparaciones y unas 16 a 22 muestra por qué conviene mantener el árbol equilibrado.

5. Conclusiones

1. Se implementó un BST de estudiantes en C++17 sin la librería estándar, con inserción, búsqueda, inorden y eliminación (hoja, un hijo y dos hijos). Las seis pruebas (P1 a P6) resultaron correctas.

2. El costo de las operaciones depende de la altura del árbol: $O(h)$, con h entre $\log_2 n$ y $n-1$.

3. Insertar códigos en orden ascendente degenera el BST en una cadena ($h = N - 1$); con $N = 100$ la altura fue 99 frente a 12,4 con inserción aleatoria.

4. Para el SIGA-UNAP, donde los códigos se asignan en orden, se necesita una estructura autoequilibrada como el AVL (Semana 10).

5. El sistema de consulta en Python mostró que el BST permite buscar, insertar, eliminar y listar en orden, con tiempos de fracciones de milisegundo para 20 registros.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Sedgewick, R., y Wayne, K. (2011). *Algorithms* (4.ª ed.). Addison-Wesley.

Zanabria Gálvez, A. H. (2026). *Semana 9: Árboles binarios de búsqueda (BST). Guía teórica ampliada y guía de práctica* [Material de curso, SIS210]. Universidad Nacional del Altiplano.